https://claude.ai/

Claude Sonnet 4.6

Q: I am a student who is a first-time learner of Python and its packages. Please provide an at-a-glance summary of how pandas has been used in the attached marimo notebooks (Wk02_Marimo_ZScore.py, Wk03_InteractivePlots_Plotly.py, Wk04_DataPreparation.py) to help me understand its usage better.

Now I have a thorough picture of all three notebooks. Let me create a clear summary document.Here's a walkthrough of how pandas builds up across your three notebooks:

Wk02 is the gentlest introduction — pandas is used just to create a tiny table from scratch (`pd.DataFrame(...)`) and save data to a CSV file (`to_csv`). You also see `.apply(lambda ...)` for the first time, which lets you run a custom rule on every row of a column.

Wk03 shifts to working with *real* data. The big new skill is `pd.read_csv(...)` to load the S&P 500 dataset. From there you learn to explore it: filter rows by condition (`df[df['column'] == value]`), find unique values (`.unique()`), and handle the very common problem of missing data (`.isna()` , `.fillna()`).

Wk04 is where pandas gets seriously powerful. You construct entirely new columns — including the Altman Z-Score (passed as whole columns into the function, which pandas handles automatically via *vectorization*) and the average cost of debt. The trickiest and most important new concept is the **lagged value**: using `.sort_values()` → `.groupby('Ticker')` → `.shift(1)` to get each company's *previous year's* debt figure. The notebook finishes with `.dropna()` to clean the data, `.apply(lambda ...)` to label rows, and `pd.crosstab()` to summarise results in a table.

The summary document above has a full table of every pandas operation used, with code examples, so you can refer back to it as you work through the notebooks.

Pandas at a Glance — Across Wk02, Wk03 & Wk04

> **What is pandas?**
> pandas is Python's go-to package for working with tabular data (think: an Excel spreadsheet, but programmable). You always start with `import pandas as pd` .

📦 Wk02 — Z-Score (Introduction to pandas)

Pandas appears briefly here, mainly to **set up a small dataset for a chart**.

What was done	Code example
Create a small DataFrame by hand	`df = pd.DataFrame({'Score': [z_score], 'Label': [ticker]})`
Add a new column with a formula	`df['Color'] = df['Score'].apply(lambda s: 'red' if s < 1.81 else ...)`
Save a DataFrame to a CSV file	`SP500_data.to_csv("sp500_raw_data.csv", index=False)`

Key ideas introduced:

- `pd.DataFrame(...)` — creates a table from a Python dictionary.
- `.apply(lambda ...)` — applies a custom rule to every row of a column.
- `.to_csv(...)` — saves your data to a file so it can be reused later.

📦 Wk03 — Interactive Plots (Reading & Exploring Data)

Pandas is used here to **load and explore** a large real-world dataset (S&P 500 companies).

What was done	Code example
Read a CSV file into a DataFrame	`df = pd.read_csv("public/sp500_raw_data_Backup.csv")`
Preview the first few rows	`df.head()`
Display the whole DataFrame (in marimo)	`df`
Get unique values in a column	`df['Sector_Key'].dropna().unique()`
Filter rows matching a condition	`df[df['Sector_Key'] == 'financial-services']`

What was done	Code example
Filter rows by another column value	`df[df['Industry_Key'] == 'credit-services']`
Control how many rows are printed	`pd.option_context('display.max_rows', None)`
Find rows with missing (NaN) values	`df[df['Total_Debt'].isna()]`
Fill in missing values	`df['Total_Debt'] = df['Total_Debt'].fillna(0)`

Key ideas introduced:

- `pd.read_csv(...)` — the most common way to load data.
- `df['column']` — selects a single column (like a list of values).
- `df[df['column'] == value]` — filters the table to matching rows only.
- `.isna()` / `.fillna()` — detect and handle missing data (`NaN`).
- `.unique()` — lists all distinct values in a column.

📦 Wk04 — Data Preparation (Building & Cleaning Variables)

This is where pandas is used most intensively — to **construct new variables, reshape data, and prepare it for analysis**.

What was done	Code example
Read CSV and fill missing values	`df = pd.read_csv(...)` then `df['Total_Debt'] = df['Total_Debt'].fillna(0)`
Create a new column using a function (vectorized)	`df['Z_Score'] = Zscore(total_assets=df['Total_Assets'], ...)`

What was done	Code example
Filter rows with/without NaN	`df[df['Z_Score'].isna()]` and `df[df['Z_Score'].notna()]`
Filter to one company and make a copy	`df_msft = df[df['Ticker'] == 'MSFT'].copy()`
Sort rows	`df_msft.sort_values('Year')`
Create a lagged (previous year's) value	`df_msft_sorted['prev_debt'] = df_msft_sorted['Total_Debt'].shift(1)`
Compute lagged values across multiple companies	`df.sort_values(['Ticker','Year']).groupby('Ticker')['Total_Debt'].shift(1)`
Arithmetic on columns to make a new variable	`df['avg_debt'] = 0.5 * (df['Total_Debt'] + df['prev_debt'])`
Drop rows with missing values in specific columns	`df.dropna(subset=['AvgCost_of_Debt', 'Z_Score_lag', 'Sector_Key'])`
Filter out outliers	`df_clean[df_clean['Z_Score_lag'] < 50]`
Label values	`tb['AvgCost_of_Debt (row)'] = tb['AvgCost_of_Debt'].apply(lambda x: ...)`

What was done	Code example
using apply + lambda	
Count unique items	`len(df_clean['Name'].unique())`
Calculate a median	`tb['AvgCost_of_Debt'].median()`
Create a contingency (cross) table	`pd.crosstab(index=..., columns=..., margins=True)`
Save the prepared data to CSV	`df.to_csv("public/sp500_ZScore_AvgCostofDebt.csv", index=False)`

Key ideas introduced:

- `.copy()` — makes a true independent copy so edits don't affect the original.
- `.sort_values(...)` — sorts rows by one or more columns.
- `.shift(1)` — shifts a column down by 1 row, giving you the *previous* row's value.
- `.groupby('Ticker')` — splits the data by company so operations (like `shift`) run separately per company.
- `.apply(lambda x: ...)` — transforms each value using a custom rule.
- `.dropna(subset=[...])` — removes rows where specific columns are missing.
- `pd.crosstab(...)` — counts how many observations fall into each combination of two categorical columns.

🗺️ The Big Picture

```
Wk02        →   Wk03              →   Wk04
_________       _______________       ___________________________________

Create small    Load a large CSV      Build new variables (Z-Score,
DataFrame       Explore & filter      avg cost of debt, lagged values)
Save to CSV     Handle NaN values     Group by company, sort, shift
                                      Clean data & create summaries
                                      Save ready-to-analyse CSV
```

Think of pandas as your **digital spreadsheet engine**: you load data in, clean it up, build new columns, filter what you need, and save the results — all in reproducible code.